# [ML] 3주차 예습과제_권지원

## Chapter 04. 분류 (Classification)

## 1. 분류(Classification)의 개요

### 지도 학습 (Supervised Learning)

- **Label(정답)** 이 있는 데이터를 이용해 학습하는 머신러닝 방식
- 입력 데이터와 정답(label)을 함께 제공하여 모델이 패턴을 학습하도록 함

### 분류(Classification)

- 지도 학습의 대표적인 유형
- 학습 데이터는 **feature(특징)와 label(결정 값, 클래스 값)** 으로 구성됨
- 학습된 모델을 이용해 **새로운 데이터의 미지의 레이블 값 예측**

즉,

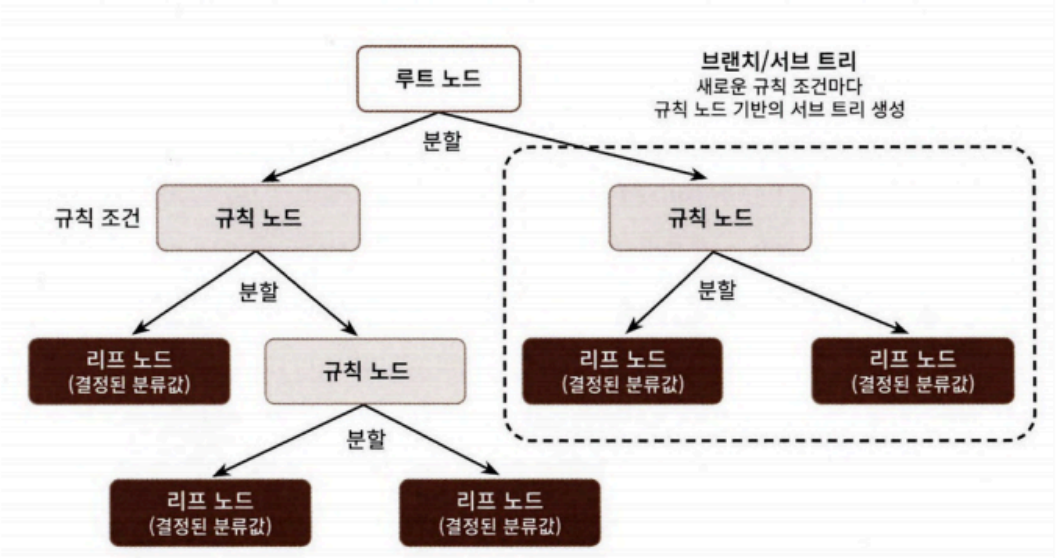
> 기존 데이터가 어떤 레이블을 가지는지의 **패턴을 알고리즘이 학습**하고
> 이를 기반으로 새로운 데이터를 분류함!

## 2. 결정 트리 (Decision Tree)

### 결정 트리 개념

결정 트리는 **데이터에 존재하는 규칙을 학습을 통해 자동으로 찾아내는 트리(Tree) 기반 분류 알고리즘**이다.

- if / else 규칙 기반 모델
- 해석이 쉽고 직관적인 구조




---

## 결정 트리의 구조

| 구성 요소 | 설명 |
|---|---|
| Decision Node (결정 노드) | 규칙 조건이 되는 노드 |
| Leaf Node (리프 노드) | 최종 클래스 값이 결정되는 노드 |

특징

- 규칙이 많아질수록 모델이 복잡해짐
- 트리 깊이가 깊어질수록 **과적합 가능성 증가**
- 결과적으로 예측 성능이 오히려 감소할 수 있음

---

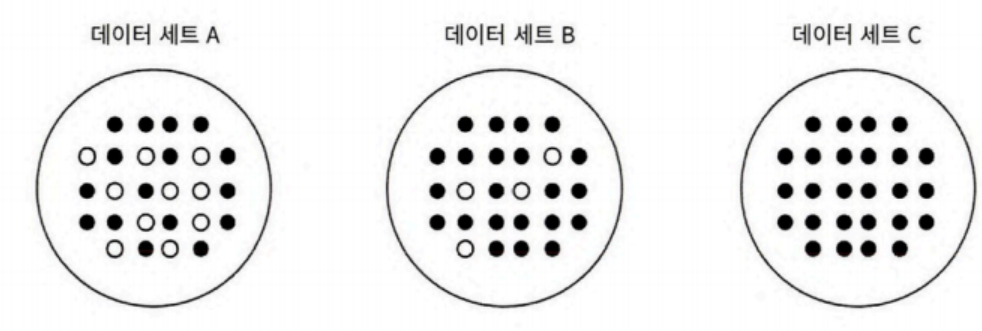
# 3. 트리의 분할 (Split)

결정 트리는 **데이터가 최대한 균일한 형태가 되도록 분할**한다.

목표

- 가능한 한 **같은 클래스의 데이터가 많이 모이도록 규칙 생성**

예시




| 데이터 상태 | 균일도 |
|---|---|
| C | 매우 높음 (모두 동일 클래스) |
| B | 중간 |
| A | 낮음 (여러 클래스 혼합) |

균일도는 **데이터 구분에 필요한 정보의 양**에 영향을 준다.

- **C** : 어떤 데이터를 뽑아도 검은 공 → 예측 쉬움
- **A** : 같은 조건에서도 여러 클래스 존재 → 더 많은 정보 필요

따라서,

> 혼잡도(impurity)가 높을수록 균일도는 낮다.

---

## 결정 트리 분할 과정

1. 균일도가 높은 속성을 기준으로 규칙 생성
2. 데이터 세트를 분할
3. 생성된 서브 데이터 세트에 대해 반복 수행

예시

레고 블록 데이터

- 노란색 블록은 모두 동그라미

규칙

```
if 색깔 == 노란색
```

---

# 4. 정보 이득 (Information Gain)과 지니 계수 (Gini Index)

데이터의 **균일도 측정 방법**

대표적으로 다음 두 가지 사용

1. 정보 이득 (Information Gain)
2. 지니 계수 (Gini Index)

---

## 엔트로피 (Entropy)

엔트로피는 **데이터 집합의 혼잡도**를 의미한다.

| 상태 | 엔트로피 |
|---|---|
| 서로 다른 값이 많이 섞임 | 높음 |
| 동일한 값이 많음 | 낮음 |

---

## 정보 이득 (Information Gain)

정보 이득은 **엔트로피 감소량**을 의미한다.

개념적으로

```
Information Gain = 1 - Entropy
```

결정 트리는 다음 기준으로 분할한다.

> **정보 이득이 높은 속성**
>
> (즉, 엔트로피가 낮고 데이터가 균일한 속성)

---

## 지니 계수 (Gini Index)

경제학의 **불평등 지수**에서 유래한 개념

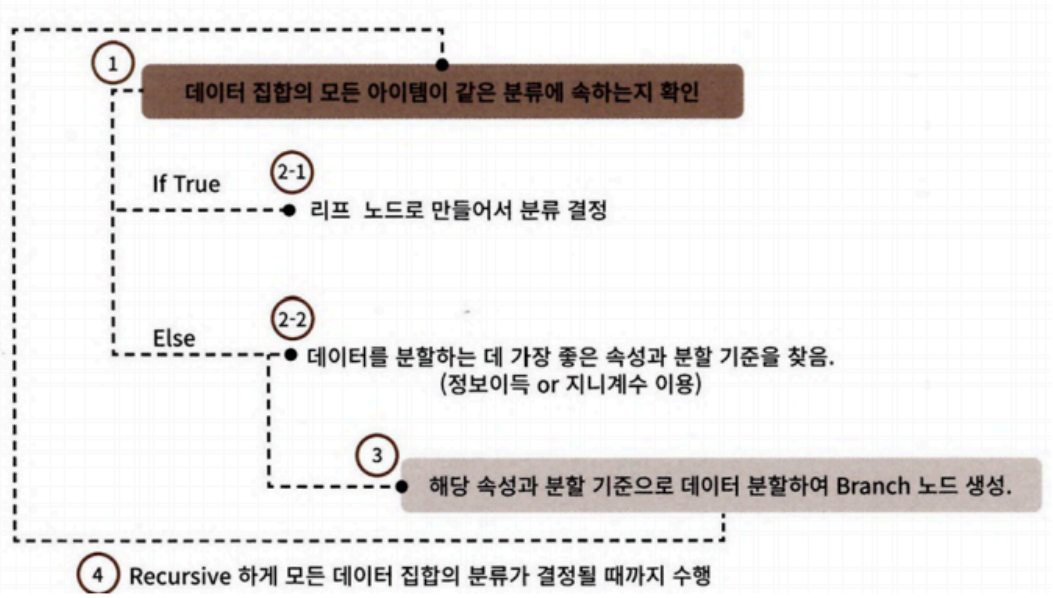
| 값 | 의미 |
|---|---|
| 0 | 완전 균등 |
| 1 | 완전 불균등 |

결정 트리에서는

> **지니 계수가 낮을수록 데이터가 균일하다고 해석**

따라서

> **지니 계수가 낮은 속성을 기준으로 분할**




---

# 5. 결정 트리 모델 특징

## 장점

- 알고리즘이 **쉽고 직관적**
- 규칙이 명확하여 **시각화 가능**
- 데이터 **스케일링이나 정규화 필요 없음**
- 균일도 기준으로 분류

---

## 단점

- **과적합(Overfitting) 발생 가능**
- feature가 많으면 트리가 매우 복잡해짐
- 깊이가 커질수록 일반화 성능 감소

따라서

> 트리 크기를 제한하는 **하이퍼파라미터 튜닝 필요**

---

# 6. 결정 트리 파라미터

사이킷런 제공 클래스

- `DecisionTreeClassifier` → 분류
- `DecisionTreeRegressor` → 회귀

사이킷런의 결정 트리는 **CART(Classification And Regression Trees)** 알고리즘 기반이다.

---

# 주요 하이퍼파라미터

| 파라미터 | 설명 |
|---|---|
| min_samples_split | 노드 분할을 위한 최소 샘플 수 |
| min_samples_leaf | 리프 노드가 가져야 할 최소 샘플 수 |
| max_features | 분할 시 고려할 최대 feature 수 |
| max_depth | 트리 최대 깊이 |
| max_leaf_nodes | 리프 노드 최대 개수 |

---

## min_samples_split

- 노드 분할에 필요한 최소 데이터 수
- 기본값: **2**

특징

- 값이 작을수록 분할이 많이 발생
- **과적합 가능성 증가**

---

## min_samples_leaf

- 리프 노드가 가져야 할 최소 데이터 수
- 값이 클수록 분할이 줄어듦
- 과적합 제어 목적

주의

> 클래스 불균형 데이터에서는 너무 크게 설정하면 문제 발생 가능

---

## max_features

분할 시 사용할 feature 수

| 설정값 | 의미 |
|---|---|
| None | 모든 feature 사용 |
| int | 지정한 개수 |
| float | 전체 feature의 비율 |
| sqrt | √(전체 feature 수) |
| log2 | log2(전체 feature 수) |
| auto | sqrt와 동일 |

---

## max_depth

트리의 최대 깊이

기본값

```
None
```

의미

- 클래스가 완전히 분리될 때까지 계속 분할
- 또는 `min_samples_split` 조건까지 분할

문제

> 깊이가 깊어질수록 **과적합 발생 가능**

---

### max_leaf_nodes

- 리프 노드의 최대 개수 제한

---

# 7. 결정 트리 시각화

결정 트리는 **Graphviz** 패키지를 이용해 시각화할 수 있다.

특징

- DOT 파일 기반 그래프 시각화
- 트리 구조를 쉽게 표현 가능

---

## Graphviz 설치

1. Graphviz Windows 버전 다운로드 및 설치
2. Python wrapper 모듈 설치
3. Jupyter Notebook 재시작

---

## export_graphviz()

학습된 모델을 **DOT 파일 형태로 출력**

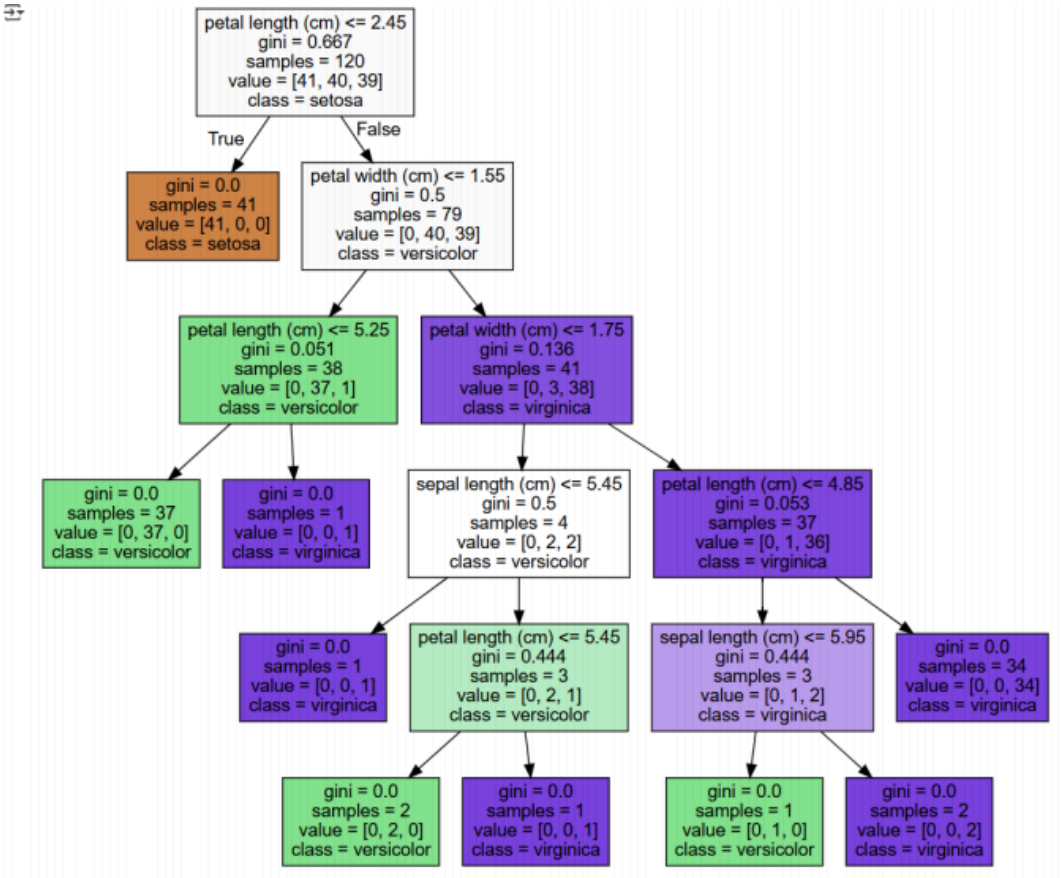
필요 입력

- 학습된 estimator
- output 파일 이름
- 클래스 이름
- feature 이름

생성된 `tree.dot` 파일을 Graphviz로 시각화 가능

---

# 8. 결정 트리 노드 해석




## 리프 노드 (Leaf Node)

- 더 이상 자식 노드가 없는 노드
- 최종 클래스 결정

리프 노드 조건

1. 하나의 클래스만 존재
2. 하이퍼파라미터 조건 충족

---

## 브랜치 노드 (Branch Node)

- 자식 노드 생성 규칙을 가진 노드

예시

```
petal_length <= 2.45
```

---

## 주요 지표

| 항목 | 의미 |
|---|---|
| gini | 지니 계수 |
| samples | 해당 노드 데이터 수 |
| value | 클래스별 데이터 개수 |

예시

```
value = [41, 40, 39]
```

- Setosa : 41
- Versicolor : 40
- Virginica : 39

---

## 노드 색상 의미

- 색이 **짙을수록 지니 계수가 낮음**
- 특정 클래스 데이터 비율이 높음

---

# 9. Feature Importance

결정 트리는 **feature 중요도**를 계산할 수 있다.

속성

```
feature_importances_
```

의미

- 분할 과정에서 해당 feature가 **얼마나 효율적으로 불순도를 감소시켰는지**

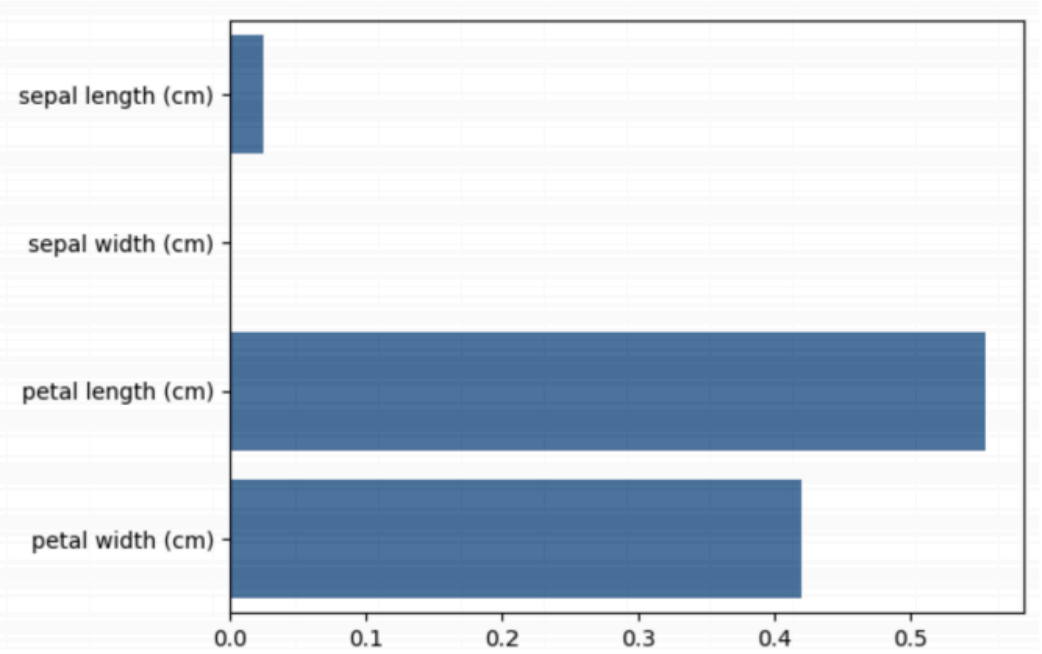
특징

- 값이 클수록 중요도 높음
- ndarray 형태로 반환

예시

붓꽃 데이터

> `petal_length` feature가 가장 높은 중요도




---

# 10. 결정 트리 과적합 (Overfitting)

분류 실습을 위해 **데이터 세트 생성**

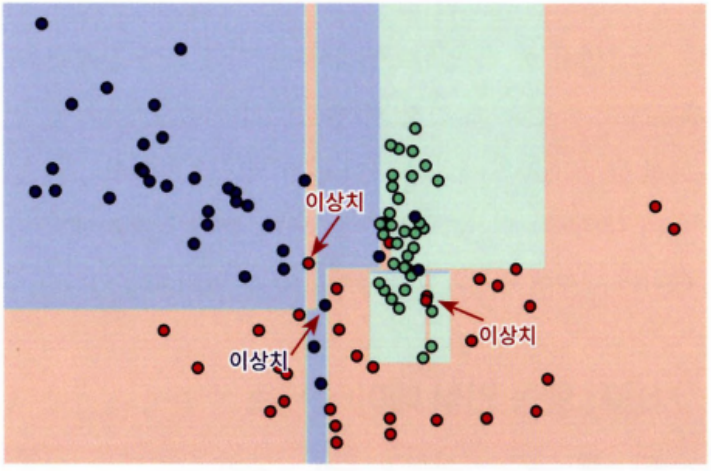
```
make_classification()
```

반환 객체

- feature 데이터
- 클래스 레이블

시각화

- X, Y 축 : feature
- 색상 : 클래스



# 11. 사용자 행동 인식 데이터 세트 실습

UCI Machine Learning Repository 제공 데이터

특징

- feature 수 : **561개**
- 공백으로 구분된 데이터

파일

- `features.txt` : feature index + feature name

데이터 특징

- 인체 움직임 관련 센서 데이터
- X, Y, Z 축 평균 및 표준편차 등

## 데이터 로딩 시 주의사항

`features.txt`에 **중복된 feature 이름 존재**

문제

> DataFrame 로딩 시 오류 발생

해결

- 중복 feature에 `_1`, `_2` 추가

예

```
feature_name
feature_name_1
feature_name_2
```

## 주요 함수

### get_new_feature_name_df()

- 중복 feature 이름 수정

| column_name | column_index |
|---|---|
| fBodyAcc-bandsEnergy()-1,16 | 3 |
| fBodyAcc-bandsEnergy()-1,24 | 3 |
| fBodyAcc-bandsEnergy()-1,8 | 3 |
| fBodyAcc-bandsEnergy()-17,24 | 3 |
| fBodyAcc-bandsEnergy()-17,32 | 3 |

### get_human_dataset()

데이터 로딩

- train feature
- train label
- test feature
- test label

설정

```
sep=' '
```

label column

```
action
```

---

# 12. 결정 트리 실습 결과

모델

```
DecisionTreeClassifier
```

하이퍼파라미터

- 기본값 사용

결과

> 정확도 약 **85.48%**

---

# 13. Tree Depth 영향 분석

결정 트리는 리프 노드가 될 때까지 계속 분할한다.

따라서

> 트리 깊이가 깊어질수록 과적합 위험 증가

---

## GridSearchCV 실험

조건

```
min_samples_split = 16
```

변화

```
max_depth
```

```
6, 8, 10, 12, 16, 20, 24
```

교차검증

```
5-fold
```

결과

> **max_depth = 8**
>
> 평균 정확도 약 **85.49%**

---

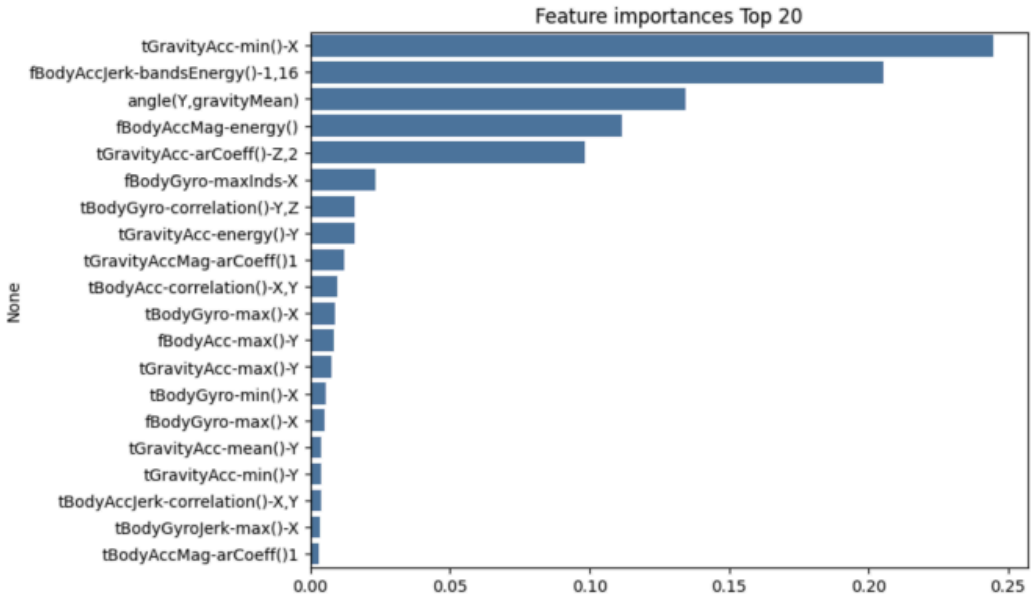
## 테스트 결과

최적 파라미터

```
max_depth = 8
min_samples_split = 16
```

테스트 정확도

> **87.17%**




---

# 14. 앙상블 학습 (Ensemble Learning)

## 개요

앙상블 학습은

> 여러 개의 분류기를 결합하여 더 정확한 예측을 만드는 방법

목표

> 단일 모델보다 **더 높은 신뢰성의 예측**

특징

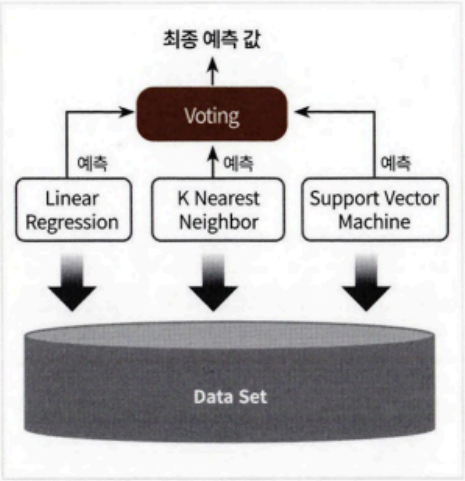
- 이미지, 영상, 음성 → 딥러닝 강점
- 정형 데이터 → 앙상블 모델 성능 우수

대표 알고리즘

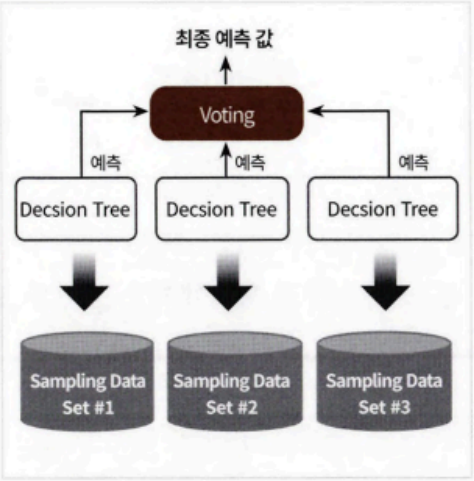
- 랜덤 포레스트
- 그래디언트 부스팅

---

# 15. 앙상블 학습 유형




Voting 방식




Bagging 방식

| 유형 | 설명 |
|---|---|
| Voting | 서로 다른 모델 결과 결합 |
| Bagging | 같은 모델 + 다른 데이터 |
| Boosting | 이전 모델 오류를 보완 |

---

## Voting

여러 알고리즘을 결합

예시

- Logistic Regression
- KNN
- SVM

---

## Bagging

특징

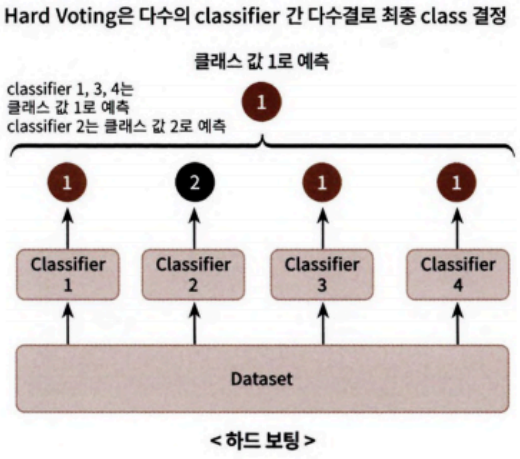
- 동일 알고리즘 사용
- 데이터 샘플링만 다름

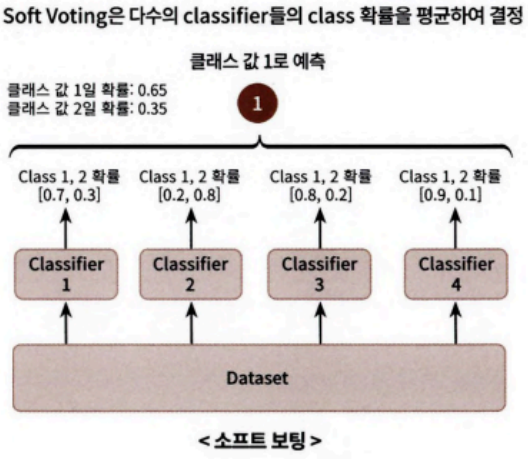
대표 알고리즘

> **Random Forest**

---

# 16. Voting 방식




## Hard Voting

다수결 방식

예

```
Classifier 1 → 1
Classifier 2 → 2
Classifier 3 → 1
Classifier 4 → 1
```

결과

```
1
```

---

## Soft Voting

각 클래스 확률 평균

예

```
Class 1 : (0.7 + 0.2 + 0.8 + 0.9) / 4 = 0.65
Class 2 : (0.3 + 0.8 + 0.2 + 0.1) / 4 = 0.35
```

결과

```
Class 1
```

일반적으로

> **Soft Voting 성능이 더 좋음**

---

## VotingClassifier

사이킷런 제공 클래스

주요 파라미터

| 파라미터 | 설명 |
|---|---|
| estimators | 여러 classifier 목록 |
| voting | hard / soft |

기본값

```
hard
```

---

# 17. 편향-분산 트레이드오프

머신러닝 모델이 해결해야 하는 핵심 문제

앙상블은

- 결정 트리의 단점 보완
- 다양한 데이터 상황 학습

결과

> **편향-분산 균형 개선**

---

# 18. 랜덤 포레스트 (Random Forest)

## 개요

랜덤 포레스트는 **Bagging 기반 알고리즘**

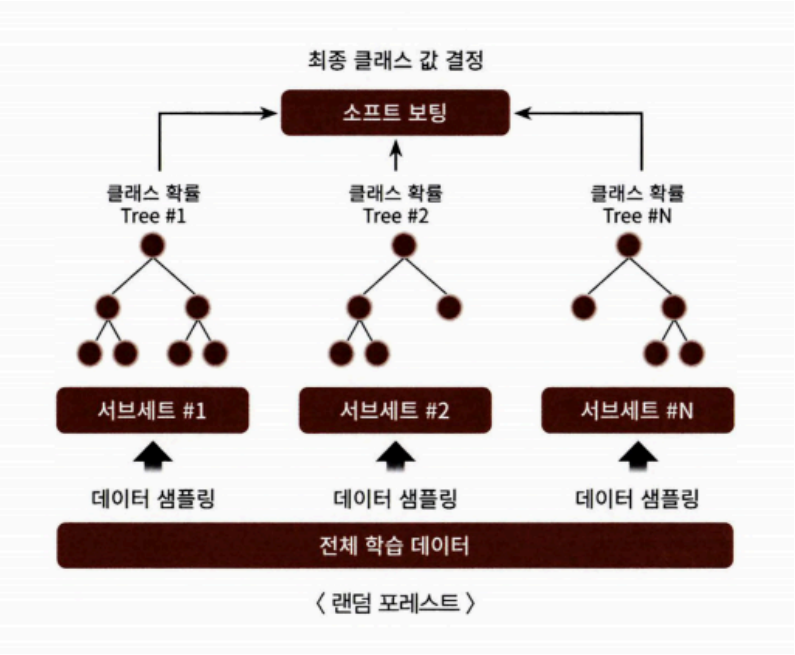
특징

- 결정 트리 여러 개 사용
- 각 트리가 독립적으로 학습
- 최종 결과는 **투표로 결정**

---

## 학습 과정

1. 전체 데이터에서 **부트스트래핑** 샘플 생성
2. 각 결정 트리 독립 학습
3. 예측 결과를 **Voting**으로 결합




---

## Bootstrapping

- 데이터 일부를 **중복 허용 샘플링**

특징

- 서브 데이터 크기 = 원본 데이터 크기
- 동일 데이터가 여러 번 포함 가능

---

# 19. RandomForestClassifier

사이킷런 제공 클래스

```
RandomForestClassifier
```

재현성 확보

```
random_state = 0
```

---

# 20. 랜덤 포레스트 주요 파라미터

| 파라미터 | 설명 |
|---|---|
| n_estimators | 결정 트리 개수 |

| 파라미터 | 설명 |
|---|---|
| max_features | 분할 시 사용할 feature |
| max_depth | 트리 깊이 |
| min_samples_leaf | 리프 최소 샘플 |
| min_samples_split | 분할 최소 샘플 |

---

### n_estimators

- 트리 개수
- 기본값

```
10
```

주의

> 트리 수 증가 → 학습 시간 증가
> 성능이 항상 좋아지는 것은 아님

---

### max_features

랜덤 포레스트 기본값

```
sqrt
```

즉

> √(전체 feature 수)

예

feature 16개

```
sqrt(16) = 4
```

---

## 21. 랜덤 포레스트 튜닝

문제

> 하이퍼파라미터가 많아 튜닝 시간이 길어짐

그래서 보통

- GridSearchCV 사용

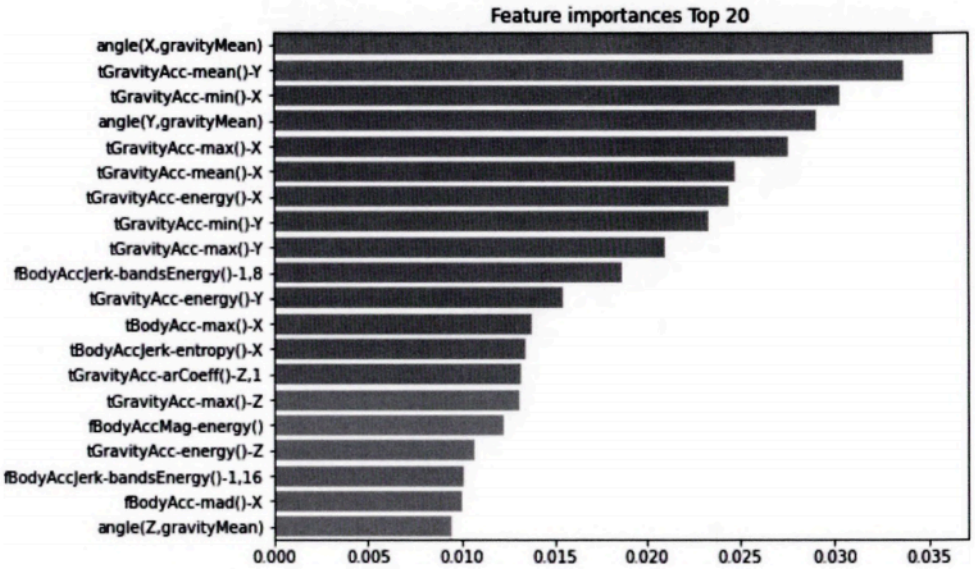
추가 옵션

```
n_jobs = -1
```

→ 모든 CPU 코어 사용

---

# 22. Feature Importance 시각화




랜덤 포레스트도

```
feature_importances_
```

속성을 통해

> 각 feature 중요도 확인 가능!

---